ANIL NEERUKONDA INSTITUTE OF TECHNOLOGY &SCIENCES (AUTONOMOUS)
(Permanent Affiliation by Andhra University & Approved by AICTE
Accredited by NBA & NAAC with A+ Grade)
Sangivalasa-531162,Bheemunipatnam
Mandal, Visakhapatnam District

A PROJECT WORK REPORT ON

BROWSER HISTORY MANAGEMENT USING STACKS

This project is submitted by

A.RAJASRI	A25126510002
B.VIVEKA VARDHINI	A25126510005
K.PRINCY	A25126510026
S.HANSIKA	A25126510048
S.DARSHNI	A25126510049

Under the Guidance of

Dr B.MEENA

Department of Computer Science and Engineering

In partial fulfillment of the requirements for the award of the

offered by

ANIL NEERUKONDA INSTITUTE OF TECHNOLOGY AND SCIENCES

Bachelor of Technology

in

COMPUTER SCIENCE AND ENGINEERING

2025-2029

ANILNEERUKONDAINSTITUTEOFTECHNOLOGY&SCIENCES
(AUTONOMOUS)
(PermanentAffiliationbyAndhraUniversity&ApprovedbyAICTEAccredited by NBA & NAAC with A+ Grade)
Sangivalasa-531162, Bheemunipatnam Mandal,
Visakhapatnam District



CERTIFICATE

This is to certify that this project work entitled **-"BROWSER HISTORY MANAGEMENT USING STACKS"** is the bona fide record work done by **Ms.A.RAJASRI, Ms.B.VIVEKA VARDHINI, Ms.K.PRINCY, Ms.S.HANSIKA, Ms.S.DARSHINI** bearing Roll No:**A25126510002, A25126510005, A25126510026,A25126510048,A25126510049** of the second year,along with batch mates submitted on the partial fulfillment of the requirement for the award of B.Tech in Computer Science and Engineering to the ANIL NEERUKONDA INSTITUTE OF TECHNOLOGY AND SCIENCES.This results embodied in this project report have not been submitted to any other board/University or Institute for the award of diploma.

LECTURER INCHARGE HEAD OF THE DEPARTMENT

ACKNOWLEDGEMENT

I would like to express my sincere gratitude to all those who helped me in completing this project successfully. First of all, I thank my project guide, DR. B. MEENA, for her valuable guidance encouragement, and continuous support throughout the project work . Her suggestions and advice helped me complete this project effectively.

I also thank the faculty members of the Department of Computer Science and Engineering, Anil Neerukonda Institute of Technology and Sciences, for providing the necessary support and resources required for the project.

I am grateful to my friends and classmates for their cooperation and helpful suggestions during the development of this project.

Finally, I thank my family for their constant encouragement, motivation and support throughout the completion of this project.

This project titled“Browser history management using stacks”helped me improve my programming knowledge and practical understanding of DSA concepts.

ABSTRACT

The **Browser History Management Using Two Stacks** project is a Data Structures and Algorithms application developed in C to simulate the back and forward navigation of a web browser. The project uses two stacks implemented using linked lists: a **Back Stack** to store previously visited pages and a **Forward Stack** to store pages available for forward navigation.

Whenever a new page is visited, the current page is pushed onto the Back Stack and the Forward Stack is cleared. When the user selects Back, the current page is moved to the Forward Stack and the most recent page from the Back Stack becomes the current page. Similarly, the Forward operation moves the current page to the Back Stack and retrieves the next page from the Forward Stack.

The project demonstrates the practical application of the **Stack data structure, linked lists, dynamic memory allocation, and LIFO (Last In, First Out) operations**. It provides a simple menu-driven interface for visiting pages, navigating backward and forward, viewing history, and clearing the browsing history.

PREFACE

Tableofcontents

Sl.No.	ChapterTitle	Sub-Contents	PageNo.
1	Introduction	• Overview of the Project • Purpose and Scope • Existing System and Proposed System	6-7
2	ProblemDefinition	• Problem Statement • Objectives of the System	8
3	SystemDesign	• System Architecture • Flowcharts • AlgorithmsUsed	9-11
4	Implementation Details	• Program Structure • Language and Environment Used • KeyFunctional Modules • Code Snippets And Screenshots	12-24
5	Results & Discussion	• Output Screens Test Case Table (Input,Expected Output, Actual Output)	25-27
6	Conclusion	• Summaryof Achievements • Limitations of the System • FutureEnhancements	28
7	References	• BooksReferred • ResearchPapers • Websites and OnlineResources	29
8	Appendix	• Source code • Source code explanation	30-43

1. INTRODUCTION

1.1 Overview of the Project

The Browser History Management Using Two Stacks project is a C-based Data Structures and Algorithms application that simulates the history navigation system of a web browser. The project uses two stacks implemented using linked lists to efficiently manage the pages visited by the user.

The Back Stack stores previously visited pages, while the Forward Stack stores pages that become available when the user moves backward. The currently opened webpage is maintained separately as the current page.

When a user visits a new webpage, the current page is pushed onto the Back Stack and the Forward Stack is cleared. When the user selects the Back operation, the current page is pushed onto the Forward Stack and the most recently visited page is retrieved from the Back Stack. Similarly, the Forward operation moves the current page to the Back Stack and retrieves the next page from the Forward Stack.

The project provides a menu-driven interface with options to visit a new page, move backward, move forward, display the current page, view complete browsing history, and clear the history. It demonstrates the practical use of stack operations such as push and pop, linked lists, dynamic memory allocation, and LIFO (Last In, First Out) principles in solving a real-world problem.

1.2 Purpose and Scope

Purpose

The main purpose of this project is to develop a Browser History Management System using two stacks and demonstrate the practical application of the Stack data structure. The project uses the LIFO (Last In, First Out) principle to efficiently manage browser navigation.
It aims to:

- Implement Back and Forward navigation using two stacks.
- Understand and apply push and pop operations.
- Demonstrate the use of linked lists and dynamic memory allocation.
- Provide an efficient way to manage previously visited webpages.
- Understand how Data Structures can be applied to a real-world browser navigation system.

Scope

The project provides a simple menu-driven system for managing browser history. It maintains a **Back Stack** for previously visited pages and a **Forward Stack** for pages available after moving backward.
The system allows the user to:

- Visit a new webpage.
- Move **Back** to a previously visited page.
- Move **Forward** to the next available page.
- Display the **current webpage**.
- View the complete **browsing history**.
- Clear the stored browsing history.

1.3 Existing System

In a traditional browser history system, visited webpages are maintained to allow users to navigate

between previously visited and recently opened pages. The Back and Forward features are commonly used to move through browsing history.
However, the internal implementation of such a system may involve complex history management mechanisms. For learning and understanding the underlying Data Structures, a simple stack-based model can be used. The existing approach can be represented using a **single or linear history list**, but managing both backward and forward navigation efficiently becomes more difficult when the user visits a new page after moving backward.
The existing system also does not clearly demonstrate the **LIFO principle and stack operations** in a simple educational form. Therefore, there is a need for a structured approach using **two stacks**, where one stack manages backward navigation and the other manages forward navigation

1.4 Proposed System

The proposed system is a Browser History Management System using Two Stacks, developed in C using the Stack data structure and linked lists. The system maintains two separate stacks: a Back Stack for previously visited webpages and a Forward Stack for pages available for forward navigation.
When a new webpage is visited, the current page is pushed onto the Back Stack and the Forward Stack is cleared. When the user selects Back, the current page is pushed onto the Forward Stack and the top page from the Back Stack becomes the current page. Similarly, the Forward operation moves the current page to the Back Stack and retrieves the top page from the Forward Stack.
The proposed system provides a simple menu-driven interface for visiting pages, Back, Forward, displaying the current page, viewing complete history, and clearing history. It demonstrates efficient stack operations and provides a clear practical understanding of how stacks can be used to implement browser navigation.

2.PROBLEM DEFINITION

2.1 Problem Statement

Managing browser history requires an efficient method to keep track of visited webpages and support Back and Forward navigation. As users browse different webpages, the system must maintain the order of visited pages and allow them to return to previously visited pages or move forward again.
The problem is to design and implement a Browser History Management System using two stacks that efficiently manages webpage navigation. The system should support operations such as visiting a new page, moving backward, moving forward, displaying the current page, viewing browsing history, and clearing history.
The project demonstrates the practical application of the Stack data structure and LIFO (Last In, First Out) principle, along with stack operations such as push and pop, to solve a real-world browser history management problem.

2.2 Objective of the System

The main objectives of the Browser History Management System using two Stacks are:

- To implement browser history management using the Stack data structure.
- To provide efficient Back and Forward navigation between visited webpages.
- To demonstrate the LIFO (Last In, First Out) principle in a real-world application.
- To perform basic stack operations such as push and pop for managing webpage history.
- To maintain and display the current webpage and browsing history.
- To handle situations such as empty history and unavailable forward or backward pages.
- To provide a simple menu-driven interface for performing browser history operations.
- To understand the practical application of Data Structures and Algorithms in browser navigation.

3.SYSTEM DESIGN

3.1 System Architecture

The Browser History Management System follows a stack-based architecture in which browser navigation is managed using stacks and a current page. The system is divided into the following main components:

1. **User Interface**

   Provides a menu-driven interface through which the user can select operations such as Visit, Back, Forward, Show Current Page, Show History, and Clear History.

2. **Browser Management Module**

   Controls the overall browser operations and maintains the current webpage. It coordinates the Back and Forward navigation.

3. **Back Stack**

   Stores previously visited webpages. When the user selects **Back**, the most recently stored page is retrieved using the pop operation.

4. **Forward Stack**

   Stores webpages that become available after moving backward. When the user selects **Forward**, the most recently stored page is retrieved.

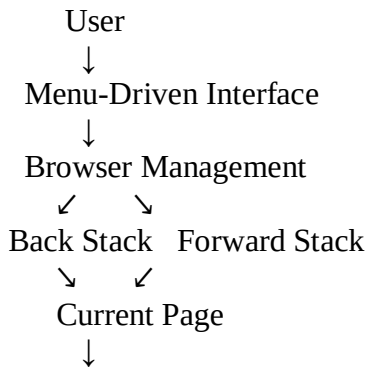
5. **Stack Operations**

   The system uses **push, pop, isEmpty, and clear** operations to manage webpage history efficiently.

6. **Linked List and Memory Management**

   Each stack is implemented using a linked list, allowing webpages to be stored dynamically using memory allocation.

Architecture Flow:



Display / Output

This architecture provides a simple and efficient way to manage browser history while demonstrating the practical use of stacks and linked lists.

3.2 Algorithm

Step 1: Start the program and initialize the Back Stack, Forward Stack, and Current Page.

Step 2: Display the menu with options to Visit New Page, Back, Forward, Show Current Page, Show Full History, Clear History, and Exit.

Step 3: Read the user's choice.

Step 4: If the user selects Visit New Page:

- Read the URL.
- Push the current page into the Back Stack.
- Set the new URL as the current page.
- Clear the Forward Stack.

Step 5: If the user selects Back:

- Check whether the Back Stack is empty.
- If it is empty, display that no previous page is available.
- Otherwise, push the current page into the Forward Stack.
- Pop the top page from the Back Stack and make it the current page.

Step 6: If the user selects Forward:

- Check whether the Forward Stack is empty.
- If it is empty, display that no forward page is available.
- Otherwise, push the current page into the Back Stack.
- Pop the top page from the Forward Stack and make it the current page.

Step 7: If the user selects Show Current Page, display the current webpage.

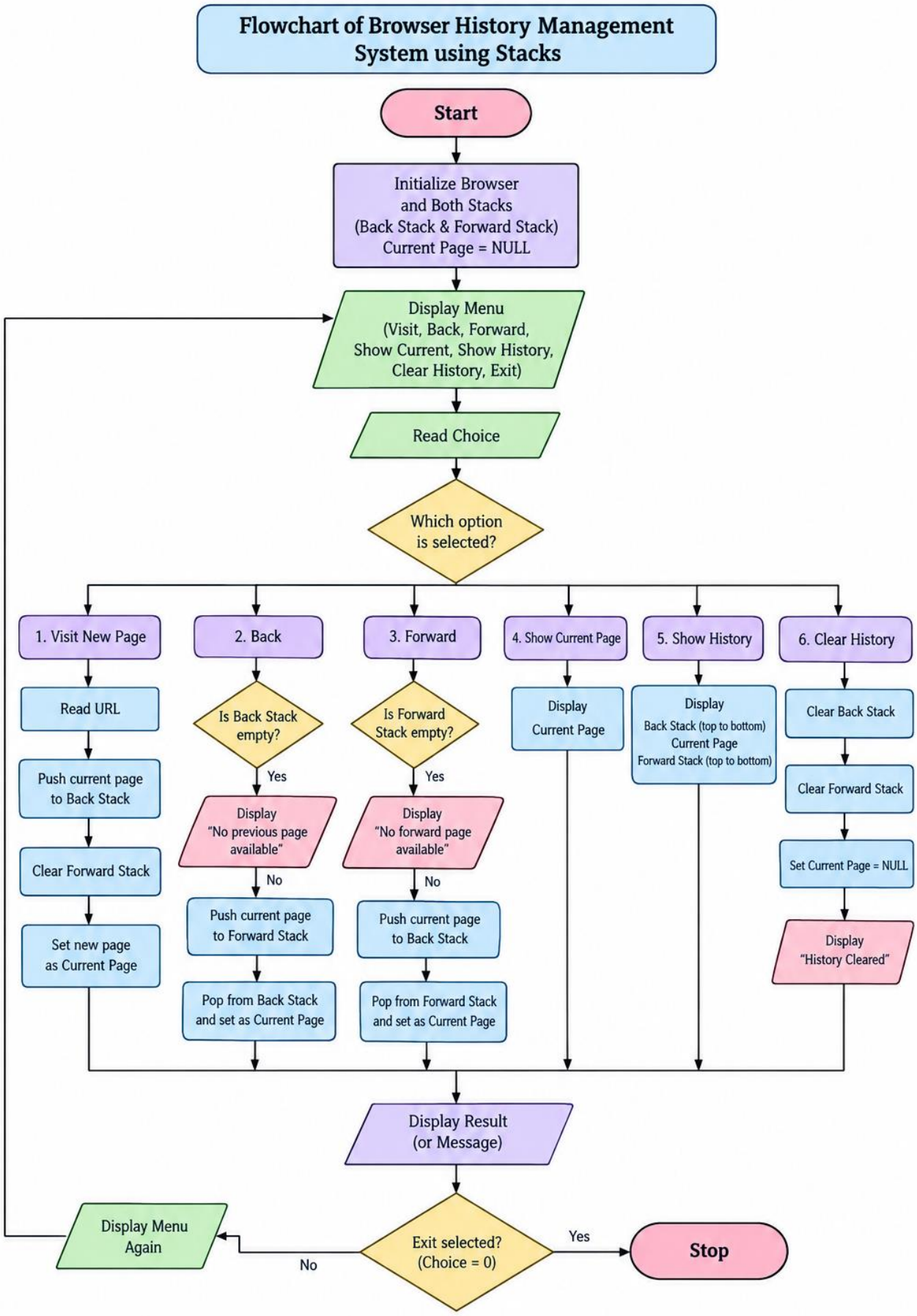
Step 8: If the user selects Show Full History, display the contents of the Back Stack, current page, and Forward Stack.

Step 9: If the user selects Clear History, clear both stacks and reset the current page.

Step 10: Repeat the menu until the user selects Exit.

Step 11: Release all dynamically allocated memory and terminate the program.

3.3 Flowcharts


Flowchart of Browser History Management System using Stacks
Start
Initialize Browser and Both Stacks (Back Stack & Forward Stack) Current Page = NULL
Display Menu (Visit, Back, Forward, Show Current, Show History, Clear History, Exit)
Read Choice
Which option is selected?
1. Visit New Page
Read URL
Push current page to Back Stack
Clear Forward Stack
Set new page as Current Page
2. Back
Is Back Stack empty?
Yes
Display "No previous page available"
No
Push current page to Forward Stack
Pop from Back Stack and set as Current Page
3. Forward
Is Forward Stack empty?
Yes
Display "No forward page available"
No
Push current page to Back Stack
Pop from Forward Stack and set as Current Page
4. Show Current Page
Display Current Page
5. Show History
Display Back Stack (top to bottom) Current Page Forward Stack (top to bottom)
6. Clear History
Clear Back Stack
Clear Forward Stack
Set Current Page = NULL
Display "History Cleared"
Display Result (or Message)
Exit selected? (Choice = 0)
Yes
Stop
No
Display Menu Again

4. IMPLEMENTATION DETAILS

4.1 Program Structure

The program is organized into different modules and functions to make the Browser History Management System using Stacks simple, modular, and easy to understand.

1. Stack Implementation
    - Defines the Node and Stack structures.
    - Implements stack operations such as push(), pop(), isEmpty(), and clearStack().

2. Browser Management
    - Defines the Browser structure containing the Back Stack, Forward Stack, and Current Page.
    - Implements visitPage(), goBack(), goForward(), showCurrent(), showHistory(), and clearHistory().

3. Input Handling
    - readInteger() handles menu choices and validates numeric input.
    - readUrl() accepts and validates webpage URLs.
    - clearInputBuffer() removes unwanted input from the buffer.

4. Main Function
    - Initializes the browser.
    - Displays the menu and accepts the user's choice.
    - Uses a switch statement to perform the selected operation.
    - Continues execution until the user selects Exit.
    - Releases dynamically allocated memory before termination.

4.2 Language and Environment Used

Programming Language:

The project is developed using the C programming language, which is suitable for implementing Data Structures and understanding memory management.

Development Environment:

The program can be developed and executed using Code::Blocks with a GCC compiler. It can also be compiled and executed through a C compiler in the command-line terminal.

Operating System:

The project is designed and tested in a Windows environment.

Data Structures Used:

- Stack – for managing browser navigation.

- Linked List – for dynamic implementation of stacks.
- Structure (struct) – for representing nodes, stacks, and browser details.

Libraries Used:

- stdio.h – used for input and output functions such as printf() and scanf().
- stdlib.h – used for dynamic memory allocation and releasing memory using malloc() and free().
- string.h – used for string operations such as strlen().

Memory Management:

The project uses dynamic memory allocation to create and delete stack nodes during program execution.

User Interface:

The system provides a console-based, menu-driven interface through which users can perform browser history operations.

Execution:

The program can be compiled, executed, and tested using a standard C compiler such as GCC or through Code::Blocks.

4.3 Key Functional Modules

The Browser History Management System consists of the following key functional modules:

1. **Stack Management Module**

   Handles the creation and management of stacks using linked lists. It performs operations such as push, pop, checking whether the stack is empty, and clearing the stack.

2. **Visit Page Module**

   Allows the user to visit a new webpage. The current page is stored in the Back Stack, the new page becomes the current page, and the Forward Stack is cleared.

3. **Back Navigation Module**

   Moves the user to the previously visited webpage by pushing the current page into the Forward Stack and popping a page from the Back Stack.

4. **Forward Navigation Module**

   Allows the user to move forward by pushing the current page into the Back Stack and popping the next page from the Forward Stack.

5. **Current Page Module**

   Displays the webpage that is currently being viewed by the user.

6. **History Display Module**

   Displays the contents of the Back Stack, current page, and Forward Stack, providing a complete view of the browsing history.

7. **History Clearing Module**

   Clears both stacks and resets the current page, allowing the user to start a new browsing session.

8. **Input Handling Module**

   Accepts and validates menu choices and URLs while handling invalid input and URLs that exceed the allowed length.

4.4 Code Snippets

Node and Stack Structure

```
struct Node {
    char url[MAX_URL];
    struct Node *next;
};

struct Stack {
    struct Node *top;
    int size;
};
```

Push Operation

```
int push(struct Stack *stack, const char *url) {
    struct Node *newNode = malloc(sizeof(*newNode));

    if (newNode == NULL) {
        return 0;
    }

    snprintf(newNode->url, sizeof(newNode->url), "%s", url);

    newNode->next = stack->top;
    stack->top = newNode;
    stack->size++;

    return 1;
}
```

Visit Page Operation

```
void visitPage(struct Browser *browser, const char *url) {
    if (browser->hasCurrent) {
        if (!push(&browser->backStack, browser->current)) {
            printf("Memory allocation failed.\n");
            return;
        }
    }

    snprintf(browser->current,
             sizeof(browser->current), "%s", url);

    browser->hasCurrent = 1;

    clearStack(&browser->forwardStack);

    printf("Visited: %s\n", browser->current);
}
```

Back Navigation

```
void goBack(struct Browser *browser) {
    char previousPage[MAX_URL];

    if (isEmpty(&browser->backStack)) {
        printf("No previous page to go back to.\n");
        return;
    }

    if (!push(&browser->forwardStack, browser->current)) {
        printf("Memory allocation failed.\n");
        return;
    }

    if (pop(&browser->backStack,
            previousPage,
            sizeof(previousPage))) {

        snprintf(browser->current,
                 sizeof(browser->current),
                 "%s", previousPage);

        printf("Went back to: %s\n", browser->current);
    }
}
```

Forward Navigation

```
void goForward(struct Browser *browser) {
    char nextPage[MAX_URL];

    if (isEmpty(&browser->forwardStack)) {
```

```
        printf("No forward page available.\n");
        return;
    }

    if (!push(&browser->backStack, browser->current)) {
        printf("Memory allocation failed.\n");
        return;
    }

    if (pop(&browser->forwardStack,
            nextPage,
            sizeof(nextPage))) {

        snprintf(browser->current,
                 sizeof(browser->current),
                 "%s", nextPage);

        printf("Went forward to: %s\n", browser->current);
    }
}
```

Menu-Driven Switch Case

```
switch (choice) {
    case 1:
        if (readUrl(url, sizeof(url))) {
            visitPage(&browser, url);
        }
        break;

    case 2:
        goBack(&browser);
        break;

    case 3:
        goForward(&browser);
        break;

    case 4:
        showCurrent(&browser);
        break;

    case 5:
        showHistory(&browser);
        break;

    case 6:
        clearHistory(&browser);
        break;

    case 0:
```

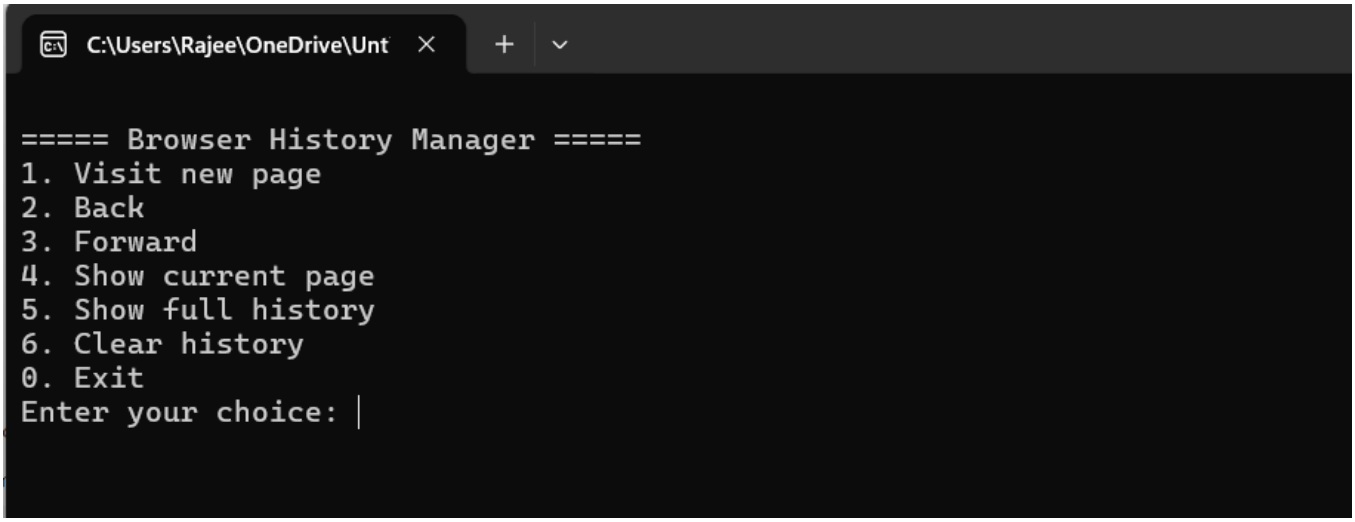
```
        printf("Exiting program.\n");
        break;

    default:
        printf("Invalid choice. Please try again.\n");
}
```

4.5 Screenshots

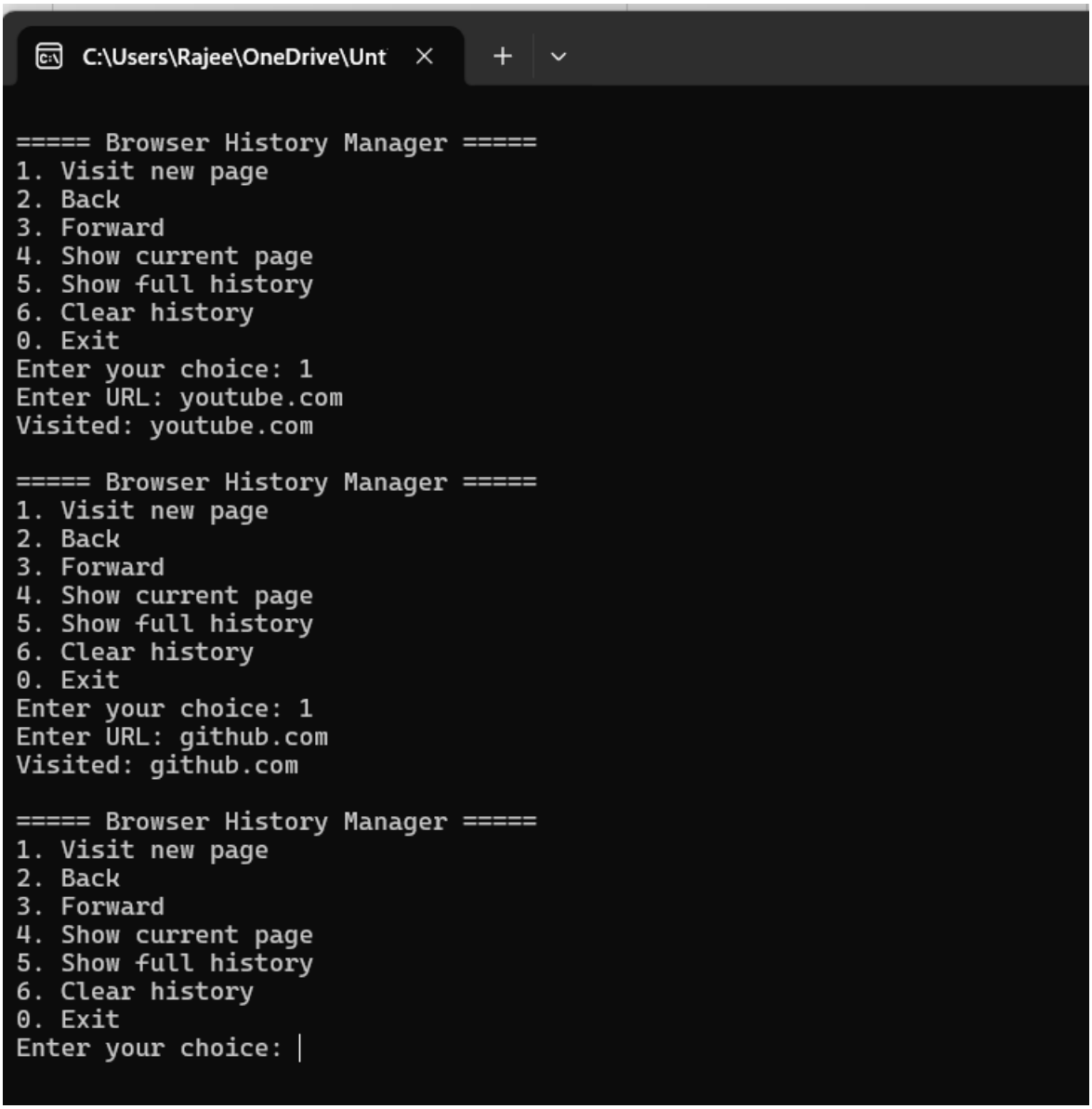
Main Menu

The main menu provides the user with different options to manage browser history. It allows the user to visit a new page, navigate backward or forward, view the current page, display complete history, clear history, and exit the program.



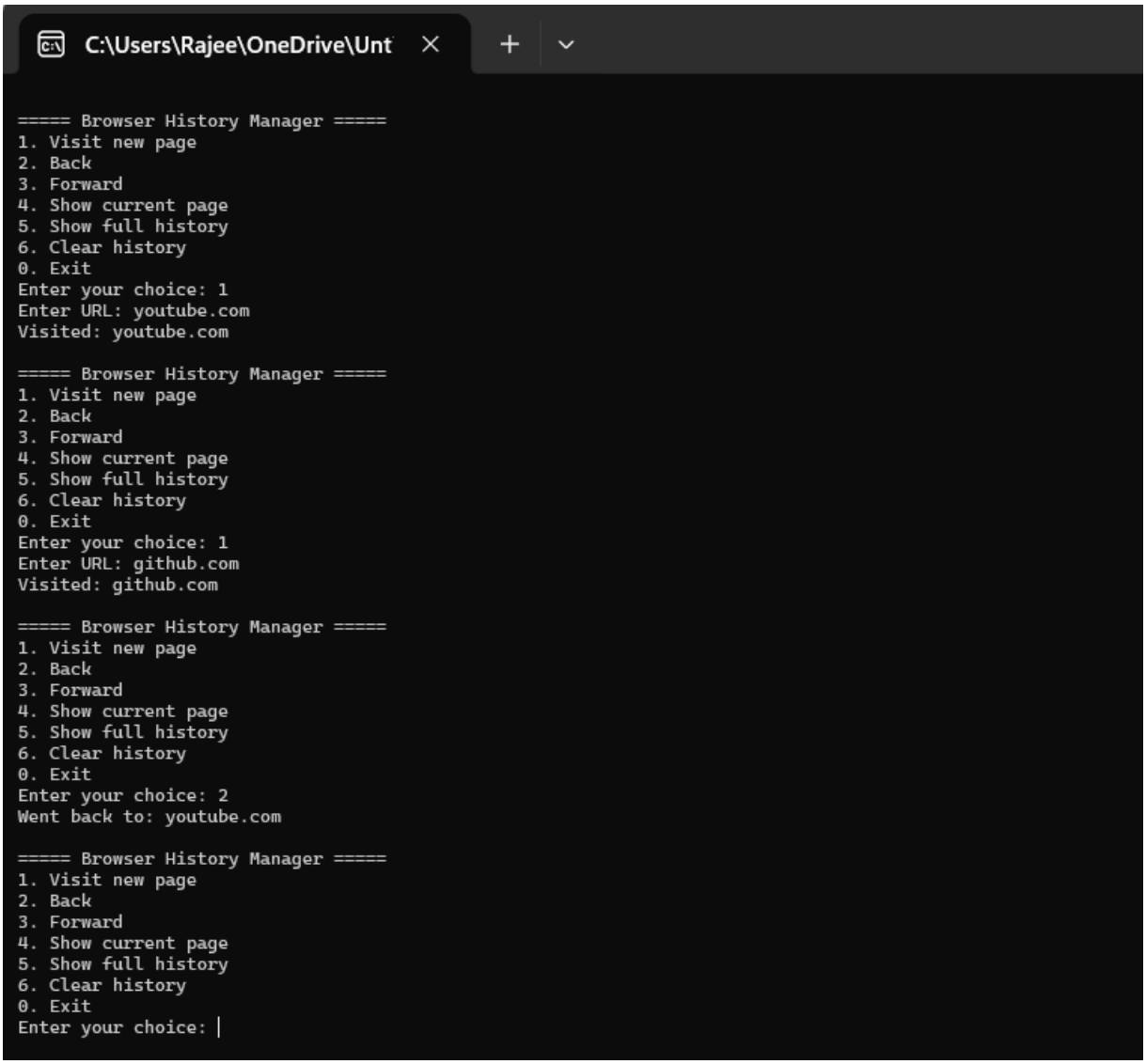
Visit New Page

This screenshot shows the process of visiting a new webpage. The user enters a URL, and the program stores the previous current page in the Back Stack before opening the new page.



Back Operation

The Back operation allows the user to move to the previously visited webpage. The current page is moved to the Forward Stack, and the top page from the Back Stack becomes the current page.



Forward Operation

The Forward operation allows the user to return to a webpage that was previously available after performing the Back operation. The current page is moved to the Back Stack, and the top page from the Forward Stack becomes the current page.



Show Current Page

This screenshot displays the webpage that is currently open. The program retrieves and displays the current page maintained by the browser history system.



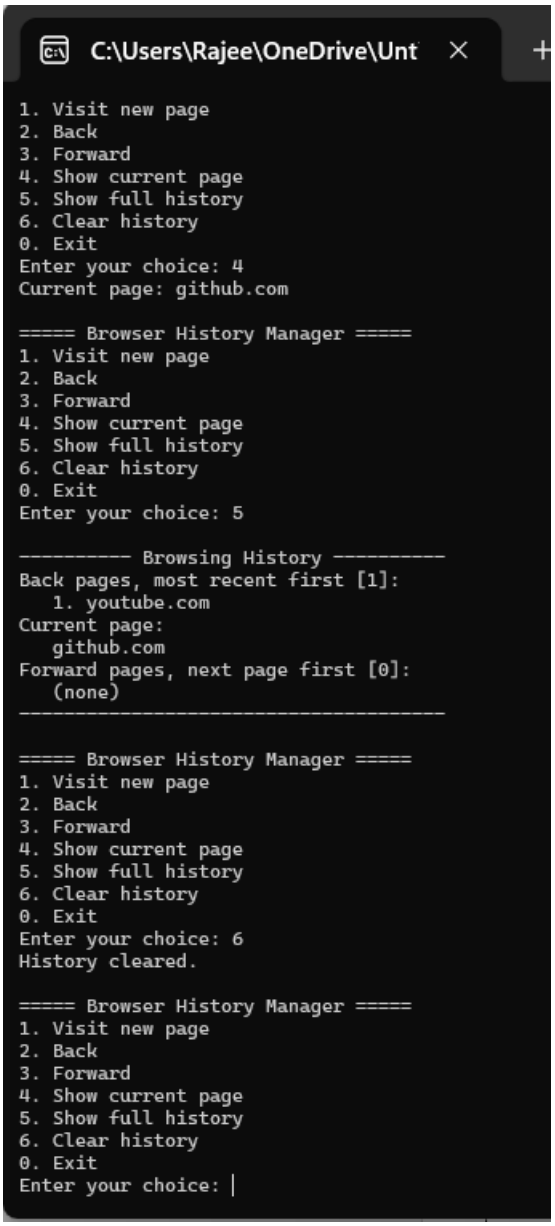
Show Full History

This screenshot displays the complete browser history. It shows the pages stored in the Back Stack, the Current Page, and the pages available in the Forward Stack.



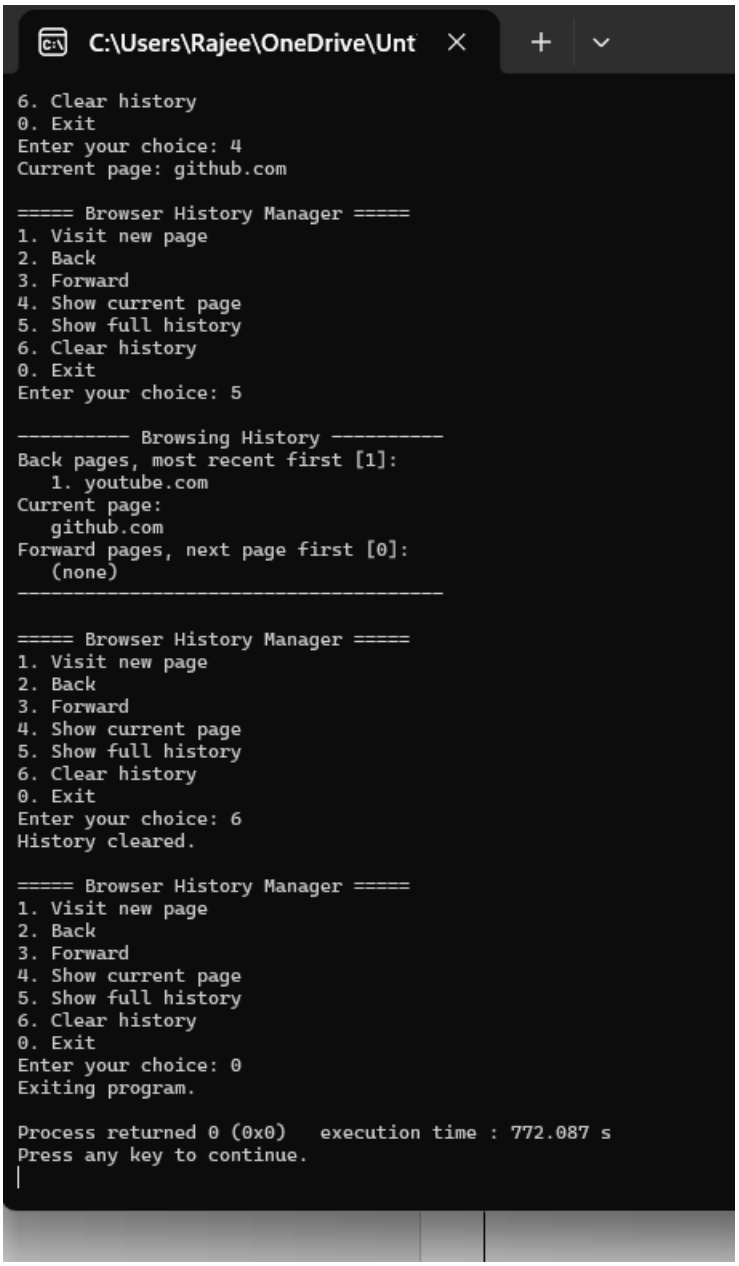
Clear History

The Clear History operation removes all webpages stored in the Back Stack and Forward Stack. The current page is also reset, indicating that the browser history has been cleared successfully.



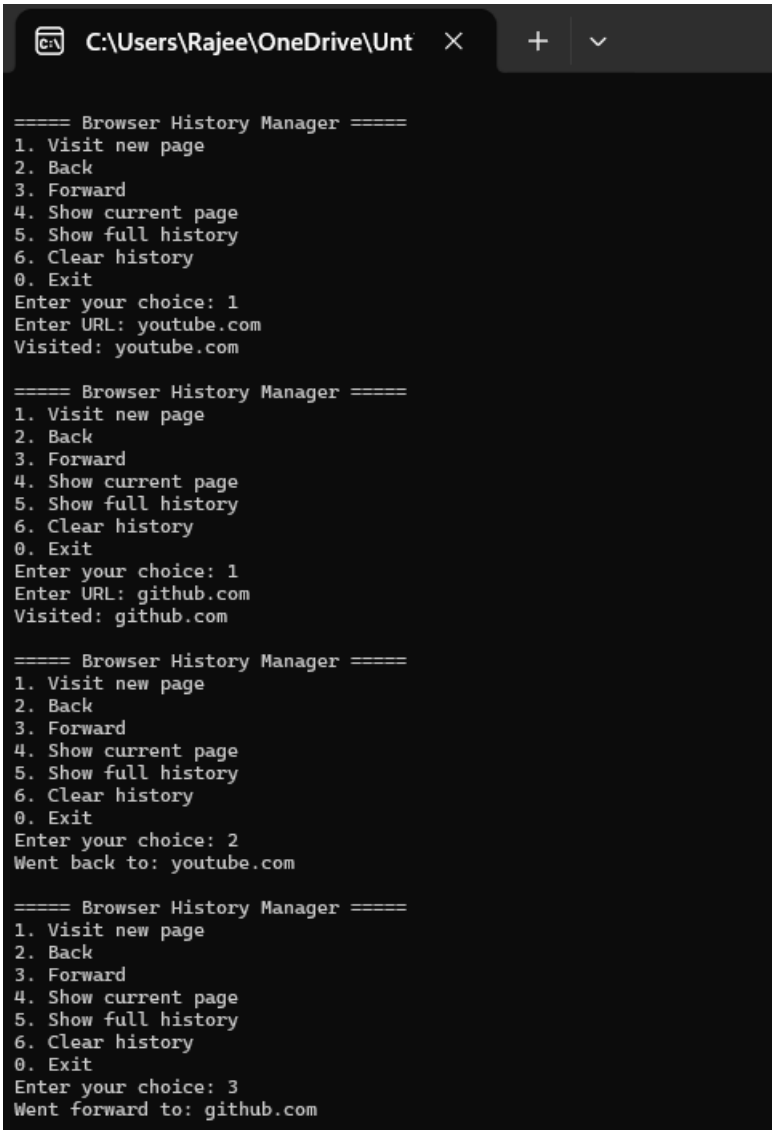
Exit

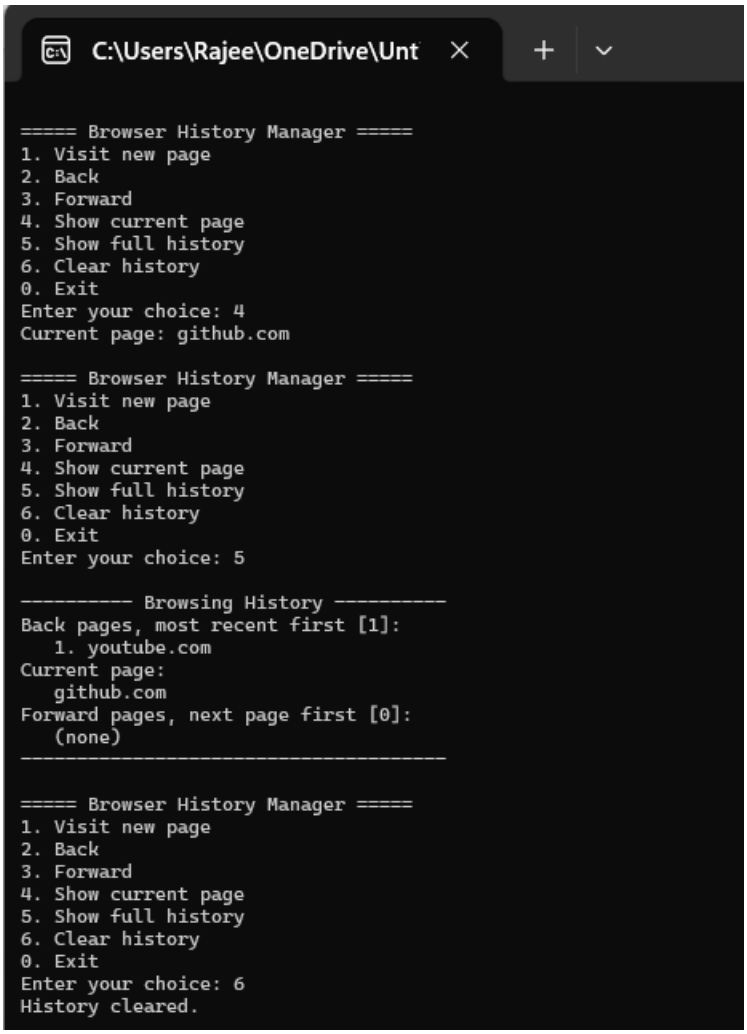
This screenshot shows the program termination message after the user selects the Exit option. The program releases the dynamically allocated memory before terminating.


C:\Users\Rajee\OneDrive\Unt
6. Clear history
0. Exit
Enter your choice: 4
Current page: github.com
===== Browser History Manager =====
1. Visit new page
2. Back
3. Forward
4. Show current page
5. Show full history
6. Clear history
0. Exit
Enter your choice: 5
---------- Browsing History ----------
Back pages, most recent first [1]:
1. youtube.com
Current page:
github.com
Forward pages, next page first [0]:
(none)

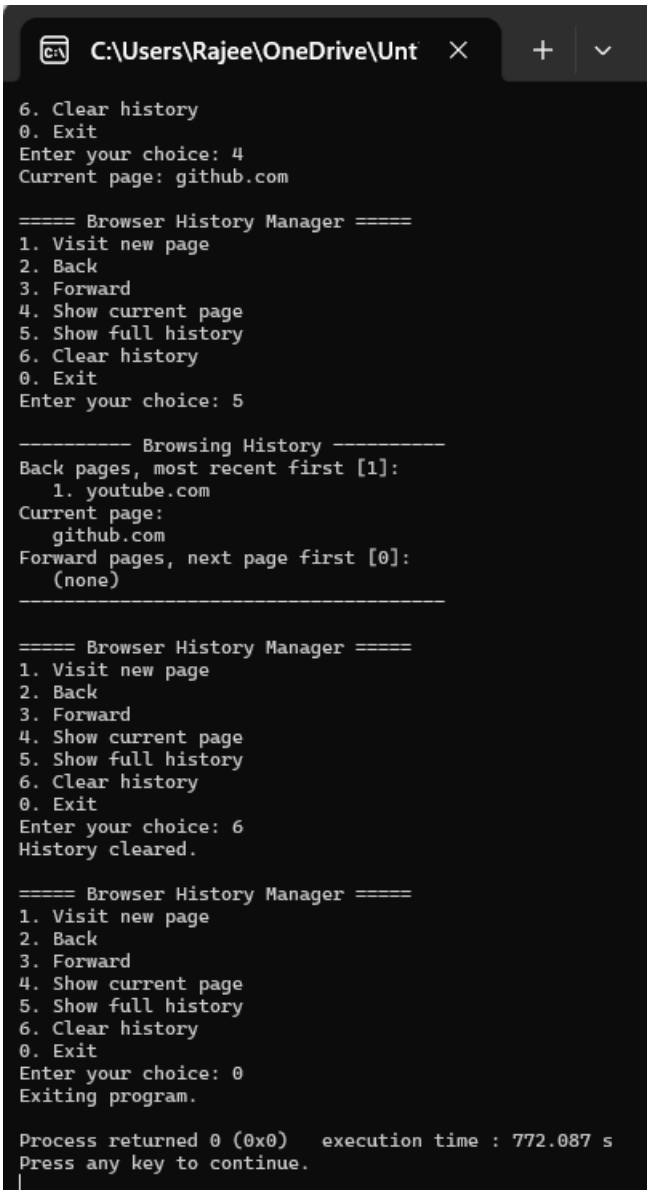
===== Browser History Manager =====
1. Visit new page
2. Back
3. Forward
4. Show current page
5. Show full history
6. Clear history
0. Exit
Enter your choice: 6
History cleared.
===== Browser History Manager =====
1. Visit new page
2. Back
3. Forward
4. Show current page
5. Show full history
6. Clear history
0. Exit
Enter your choice: 0
Exiting program.
Process returned 0 (0x0) execution time : 772.087 s
Press any key to continue.

5.OUTPUT SCREENS




C:\Users\Rajee\OneDrive\Unt
===== Browser History Manager =====
1. Visit new page
2. Back
3. Forward
4. Show current page
5. Show full history
6. Clear history
0. Exit
Enter your choice: 4
Current page: github.com
===== Browser History Manager =====
1. Visit new page
2. Back
3. Forward
4. Show current page
5. Show full history
6. Clear history
0. Exit
Enter your choice: 5
---------- Browsing History ----------
Back pages, most recent first [1]:
1. youtube.com
Current page:
github.com
Forward pages, next page first [0]:
(none)

===== Browser History Manager =====
1. Visit new page
2. Back
3. Forward
4. Show current page
5. Show full history
6. Clear history
0. Exit
Enter your choice: 6
History cleared.


C:\Users\Rajee\OneDrive\Unt
6. Clear history
0. Exit
Enter your choice: 4
Current page: github.com
===== Browser History Manager =====
1. Visit new page
2. Back
3. Forward
4. Show current page
5. Show full history
6. Clear history
0. Exit
Enter your choice: 5
---------- Browsing History ----------
Back pages, most recent first [1]:
1. youtube.com
Current page:
github.com
Forward pages, next page first [0]:
(none)

===== Browser History Manager =====
1. Visit new page
2. Back
3. Forward
4. Show current page
5. Show full history
6. Clear history
0. Exit
Enter your choice: 6
History cleared.
===== Browser History Manager =====
1. Visit new page
2. Back
3. Forward
4. Show current page
5. Show full history
6. Clear history
0. Exit
Enter your choice: 0
Exiting program.
Process returned 0 (0x0) execution time : 772.087 s
Press any key to continue.

6.CONCLUSION

The Browser History Management Using Stacks project successfully demonstrates the practical application of stack and linked list data structures in a browser navigation system. The system provides basic browser history operations such as visiting webpages, moving backward and forward, viewing the current page, displaying complete history, and clearing history. The project helps in understanding the LIFO principle and the use of dynamic memory allocation in C.

6.1 Summary of Achievements

The major achievements of the project are:

- Successfully implemented browser history management using stacks.
- Implemented Back and Forward navigation operations.
- Used linked lists for dynamic stack implementation.
- Implemented push, pop, display, and clear operations.
- Developed a simple menu-driven user interface.
- Implemented input validation and empty-stack handling.
- Demonstrated the practical application of Data Structures and Algorithms.
- Used dynamic memory allocation for efficient memory management.

6.2 Limitations of the System

Although the system performs the required browser history operations, it has some limitations:

- It is a console-based application and does not provide a graphical user interface.
- It does not connect to or open actual websites.
- Browsing history is stored only during program execution.
- The system supports only a single browsing session.
- Features such as bookmarks, multiple tabs, and search history are not included.
- Advanced browser functionalities are not implemented.

6.3 Future Enhancements

The system can be improved in the future by adding several advanced features:

- Develop a graphical user interface for better user interaction.
- Add support for multiple browser tabs.
- Implement bookmark management.
- Store browsing history permanently using files or a database.
- Add timestamps for visited webpages.
- Provide a search option for browsing history.
- Add history import and export functionality.
- Implement webpage visit statistics and usage analysis.

7.REFERENCES

1. E. Balagurusamy, *Programming in ANSI C*, McGraw Hill Education.
2. Reema Thareja, *Data Structures Using C*, Oxford University Press.
3. Ellis Horowitz, Sartaj Sahni and Susan Anderson-Freed, *Fundamentals of Data Structures in C*, Universities Press.
4. Yashavant Kanetkar, *Let Us C*, BPB Publications.
5. TutorialsPoint, "Data Structures – Stack," TutorialsPoint.
6. GeeksforGeeks, "Stack Data Structure," GeeksforGeeks.
7. GeeksforGeeks, "Linked List Data Structure," GeeksforGeeks.
8. Programiz, "Stack Data Structure," Programiz.
9. C Programming Documentation and Standard Library references for stdio.h, stdlib.h, and string.h.
10. Class notes and laboratory materials related to Data Structures and Algorithms.

8.APPENDIX

8.1 SOURCE CODE

```
#include <stdio.h>
#include <stdlib.h>
#include <string.h>

#define MAX_URL 100

struct Node {
    char url[MAX_URL];
    struct Node *next;
};

struct Stack {
    struct Node *top;
    int size;
};

void initStack(struct Stack *stack) {
    stack->top = NULL;
    stack->size = 0;
}

int isEmpty(const struct Stack *stack) {
    return stack->top == NULL;
}

int push(struct Stack *stack, const char *url) {
    struct Node *newNode = malloc(sizeof(*newNode));

    if (newNode == NULL) {
        return 0;
    }


    snprintf(newNode->url, sizeof(newNode->url), "%s", url);

    newNode->next = stack->top;
    stack->top = newNode;
    stack->size++;

    return 1;
}


int pop(struct Stack *stack, char *output, size_t outputSize) {
    struct Node *temporary;
```

```
    if (isEmpty(stack)) {
        return 0;
    }

    temporary = stack->top;

    if (outputSize > 0) {
        snprintf(output, outputSize, "%s", temporary->url);
    }

    stack->top = temporary->next;
    stack->size--;

    free(temporary);

    return 1;
}

void clearStack(struct Stack *stack) {
    char unused[MAX_URL];

    while (pop(stack, unused, sizeof(unused))) {

    }
}


void printStack(const struct Stack *stack) {
    const struct Node *currentNode;
    int position = 1;

    if (isEmpty(stack)) {
        printf("    (none)\n");
        return;
    }

    currentNode = stack->top;

    while (currentNode != NULL) {
        printf("    %d. %s\n", position, currentNode->url);
        currentNode = currentNode->next;
        position++;
    }
}


struct Browser {
    struct Stack backStack;
    struct Stack forwardStack;
    char current[MAX_URL];
    int hasCurrent;
```

```
};

void initBrowser(struct Browser *browser) {
    initStack(&browser->backStack);
    initStack(&browser->forwardStack);

    browser->current[0] = '\0';
    browser->hasCurrent = 0;
}


void visitPage(struct Browser *browser, const char *url) {
    if (browser->hasCurrent) {
        if (!push(&browser->backStack, browser->current)) {
            printf("Memory allocation failed. Page was not visited.\n");
            return;
        }
    }

    snprintf(browser->current, sizeof(browser->current), "%s", url);
    browser->hasCurrent = 1;


    clearStack(&browser->forwardStack);

    printf("Visited: %s\n", browser->current);
}

void goBack(struct Browser *browser) {
    char previousPage[MAX_URL];

    if (isEmpty(&browser->backStack)) {
        printf("No previous page to go back to.\n");
        return;
    }


    if (!push(&browser->forwardStack, browser->current)) {
        printf("Memory allocation failed. Cannot go back.\n");
        return;
    }

    if (pop(&browser->backStack,
            previousPage,
            sizeof(previousPage))) {
        snprintf(browser->current,
                 sizeof(browser->current),
                 "%s",
                 previousPage);

        printf("Went back to: %s\n", browser->current);
```

```
    }
}


void goForward(struct Browser *browser) {
    char nextPage[MAX_URL];

    if (isEmpty(&browser->forwardStack)) {
        printf("No forward page available.\n");
        return;
    }


    if (!push(&browser->backStack, browser->current)) {
        printf("Memory allocation failed. Cannot go forward.\n");
        return;
    }


    if (pop(&browser->forwardStack,
            nextPage,
            sizeof(nextPage))) {
        snprintf(browser->current,
                 sizeof(browser->current),
                 "%s",
                 nextPage);

        printf("Went forward to: %s\n", browser->current);
    }
}

void showCurrent(const struct Browser *browser) {
    if (!browser->hasCurrent) {
        printf("No page is open yet.\n");
    } else {
        printf("Current page: %s\n", browser->current);
    }
}

void showHistory(const struct Browser *browser) {
    printf("\n---------- Browsing History ----------\n");

    printf("Back pages, most recent first [%d]:\n",
           browser->backStack.size);
    printStack(&browser->backStack);

    printf("Current page:\n");
    if (browser->hasCurrent) {
        printf("   %s\n", browser->current);
    } else {
        printf("   (none)\n");
```

```
    }

    printf("Forward pages, next page first [%d]:\n",
           browser->forwardStack.size);
    printStack(&browser->forwardStack);

    printf("---------------------------------------\n");
}

void clearHistory(struct Browser *browser) {
    clearStack(&browser->backStack);
    clearStack(&browser->forwardStack);

    browser->current[0] = '\0';
    browser->hasCurrent = 0;

    printf("History cleared.\n");
}

void clearInputBuffer(void) {
    int character;

    while ((character = getchar()) != '\n' &&
           character != EOF) {
        /* Discard remaining input. */
    }
}

int readInteger(const char *prompt) {
    int value;

    while (1) {
        printf("%s", prompt);

        if (scanf("%d", &value) == 1) {
            clearInputBuffer();
            return value;
        }

        printf("Invalid input. Please enter a number.\n");
        clearInputBuffer();
    }
}


int readUrl(char *destination, size_t destinationSize) {
    size_t length;

    while (1) {
        printf("Enter URL: ");
```

```
        if (fgets(destination, destinationSize, stdin) == NULL) {
            destination[0] = '\0';
            return 0;
        }

        length = strlen(destination);


        if (length == 0 || destination[length - 1] != '\n') {
            clearInputBuffer();
            printf("URL is too long. Maximum length is %zu characters.\n",
                   destinationSize - 1);
            continue;
        }

        destination[length - 1] = '\0';

        if (destination[0] == '\0') {
            printf("URL cannot be empty.\n");
            continue;
        }

        return 1;
    }
}



int main(void) {
    struct Browser browser;
    char url[MAX_URL];
    int choice;

    initBrowser(&browser);

    do {
        printf("\n===== Browser History Manager =====\n");
        printf("1. Visit new page\n");
        printf("2. Back\n");
        printf("3. Forward\n");
        printf("4. Show current page\n");
        printf("5. Show full history\n");
        printf("6. Clear history\n");
        printf("0. Exit\n");

        choice = readInteger("Enter your choice: ");

        switch (choice) {
            case 1:
                if (readUrl(url, sizeof(url))) {
                    visitPage(&browser, url);
```

```
            }
            break;

        case 2:
            goBack(&browser);
            break;

        case 3:
            goForward(&browser);
            break;

        case 4:
            showCurrent(&browser);
            break;

        case 5:
            showHistory(&browser);
            break;

        case 6:
            clearHistory(&browser);
            break;

        case 0:
            printf("Exiting program.\n");
            break;

        default:
            printf("Invalid choice. Please try again.\n");
      }

   } while (choice != 0);


   clearStack(&browser.backStack);
   clearStack(&browser.forwardStack);

   return 0;
}
```

8.2 SOURCE CODE EXPLANATION

The Browser History Management System is implemented in C using Stack and Linked List data structures. The program uses two stacks to manage Back and Forward navigation and maintains the currently opened webpage separately.

Header Files and Constant

The following header files are used for input/output, dynamic memory allocation, and string operations.

```
#include <stdio.h>
#include <stdlib.h>
```

```
#include <string.h>

#define MAX_URL 100
```

- stdio.h provides input and output functions.
- stdlib.h provides malloc() and free().
- string.h provides string handling functions.
- MAX_URL defines the maximum URL size as 100 characters.

Node Structure

Each webpage is stored in a node of the linked list.

```
struct Node {
    char url[MAX_URL];
    struct Node *next;
};
```

The url stores the webpage address and next points to the next node.

Stack Structure

The stack structure maintains the top node and the number of elements.

```
struct Stack {
    struct Node *top;
    int size;
};
```

The top pointer identifies the top element of the stack, while size stores the number of webpages.

Stack Initialization

The initStack() function initializes an empty stack.

```
void initStack(struct Stack *stack) {
    stack->top = NULL;
    stack->size = 0;
}
```

Initially, the stack has no nodes, so top is set to NULL and size is set to zero.

Push Operation

The push() function inserts a webpage at the top of the stack.

```
int push(struct Stack *stack, const char *url) {
    struct Node *newNode = malloc(sizeof(*newNode));

    if (newNode == NULL) {
        return 0;
    }

    snprintf(newNode->url, sizeof(newNode->url), "%s", url);

    newNode->next = stack->top;
    stack->top = newNode;
    stack->size++;
```

```
    return 1;
}
```
A new node is dynamically created using malloc(). The URL is stored in the node, and the new node is placed at the top of the stack.

Pop Operation

The pop() function removes the top webpage from the stack.

```
int pop(struct Stack *stack, char *output, size_t outputSize) {
    struct Node *temporary;

    if (isEmpty(stack)) {
        return 0;
    }

    temporary = stack->top;

    if (outputSize > 0) {
        snprintf(output, outputSize, "%s", temporary->url);
    }

    stack->top = temporary->next;
    stack->size--;

    free(temporary);

    return 1;
}
```
The top node is removed and its memory is released using free().

Browser Structure

The Browser structure maintains the Back Stack, Forward Stack, and current webpage.

```
struct Browser {
    struct Stack backStack;
    struct Stack forwardStack;
    char current[MAX_URL];
    int hasCurrent;
};
```
- backStack stores previously visited webpages.
- forwardStack stores webpages available for Forward navigation.
- current stores the current webpage.
- hasCurrent indicates whether a webpage is currently open.

Visit New Page

The visitPage() function handles the visiting of a new webpage.

```
void visitPage(struct Browser *browser, const char *url) {
    if (browser->hasCurrent) {
        if (!push(&browser->backStack, browser->current)) {
            printf("Memory allocation failed. Page was not visited.\n");
```

```
            return;
        }
    }

    snprintf(browser->current,
             sizeof(browser->current), "%s", url);

    browser->hasCurrent = 1;

    clearStack(&browser->forwardStack);

    printf("Visited: %s\n", browser->current);
}
```

When a new page is visited, the previous current page is pushed into the Back Stack. The new URL becomes the current page, and the Forward Stack is cleared.

Back Operation

The goBack() function performs backward navigation.

```
void goBack(struct Browser *browser) {
    char previousPage[MAX_URL];

    if (isEmpty(&browser->backStack)) {
        printf("No previous page to go back to.\n");
        return;
    }

    if (!push(&browser->forwardStack, browser->current)) {
        printf("Memory allocation failed. Cannot go back.\n");
        return;
    }

    if (pop(&browser->backStack,
            previousPage,
            sizeof(previousPage))) {

        snprintf(browser->current,
                 sizeof(browser->current),
                 "%s",
                 previousPage);

        printf("Went back to: %s\n", browser->current);
    }
}
```

The current page is moved to the Forward Stack, and the most recently stored page from the Back Stack becomes the current page.

Forward Operation

The goForward() function performs forward navigation.

```
void goForward(struct Browser *browser) {
```

```
    char nextPage[MAX_URL];

    if (isEmpty(&browser->forwardStack)) {
        printf("No forward page available.\n");
        return;
    }

    if (!push(&browser->backStack, browser->current)) {
        printf("Memory allocation failed. Cannot go forward.\n");
        return;
    }

    if (pop(&browser->forwardStack,
            nextPage,
            sizeof(nextPage))) {

        snprintf(browser->current,
                 sizeof(browser->current),
                 "%s",
                 nextPage);

        printf("Went forward to: %s\n", browser->current);
    }
}
```

The current page is pushed into the Back Stack, and the top page of the Forward Stack becomes the current page.

Display Current Page

The showCurrent() function displays the current webpage.

```
void showCurrent(const struct Browser *browser) {
    if (!browser->hasCurrent) {
        printf("No page is open yet.\n");
    } else {
        printf("Current page: %s\n", browser->current);
    }
}
```

If no webpage has been visited, the program displays an appropriate message.

Display Full History

The showHistory() function displays the Back Stack, Current Page, and Forward Stack.

```
void showHistory(const struct Browser *browser) {
    printf("\n---------- Browsing History ----------\n");

    printf("Back pages, most recent first [%d]:\n",
           browser->backStack.size);

    printStack(&browser->backStack);

    printf("Current page:\n");
```

```
    if (browser->hasCurrent) {
        printf("   %s\n", browser->current);
    } else {
        printf("   (none)\n");
    }

    printf("Forward pages, next page first [%d]:\n",
           browser->forwardStack.size);

    printStack(&browser->forwardStack);

    printf("---------------------------------------\n");
}
```

This function provides a complete view of the browser's current history state.

Clear History

The clearHistory() function removes all browsing history.

```
void clearHistory(struct Browser *browser) {
    clearStack(&browser->backStack);
    clearStack(&browser->forwardStack);

    browser->current[0] = '\0';
    browser->hasCurrent = 0;

    printf("History cleared.\n");
}
```

Both stacks are cleared and the current webpage is reset.

URL Input Validation

The readUrl() function accepts and validates the webpage URL.

```
int readUrl(char *destination, size_t destinationSize) {
    size_t length;

    while (1) {
        printf("Enter URL: ");

        if (fgets(destination, destinationSize, stdin) == NULL) {
            destination[0] = '\0';
            return 0;
        }

        length = strlen(destination);

        if (length == 0 || destination[length - 1] != '\n') {
            clearInputBuffer();

            printf("URL is too long. Maximum length is %zu characters.\n",
                   destinationSize - 1);
```

```
            continue;
        }

        destination[length - 1] = '\0';

        if (destination[0] == '\0') {
            printf("URL cannot be empty.\n");
            continue;
        }

        return 1;
    }
}
```

This function prevents empty URLs and URLs exceeding the allowed length.

Menu-Driven Main Function

The main() function controls the complete execution of the program.

```
do {
    printf("\n===== Browser History Manager =====\n");
    printf("1. Visit new page\n");
    printf("2. Back\n");
    printf("3. Forward\n");
    printf("4. Show current page\n");
    printf("5. Show full history\n");
    printf("6. Clear history\n");
    printf("0. Exit\n");

    choice = readInteger("Enter your choice: ");

    switch (choice) {
        case 1:
            if (readUrl(url, sizeof(url))) {
                visitPage(&browser, url);
            }
            break;

        case 2:
            goBack(&browser);
            break;

        case 3:
            goForward(&browser);
            break;

        case 4:
            showCurrent(&browser);
            break;

        case 5:
```

```
            showHistory(&browser);
            break;

        case 6:
            clearHistory(&browser);
            break;

        case 0:
            printf("Exiting program.\n");
            break;

        default:
            printf("Invalid choice. Please try again.\n");
    }

} while (choice != 0);
```

The switch-case statement executes the operation selected by the user. The menu continues to appear until the user selects option 0.

8.3 Memory Deallocation

Before the program terminates, all dynamically allocated nodes are removed.

```
clearStack(&browser.backStack);
clearStack(&browser.forwardStack);

return 0;
```

This prevents memory leaks and ensures that dynamically allocated memory is properly released.